Lab 17 – AI for Data Processing

Automated Data Cleaning, Preprocessing & Feature Engineering

Name: D.POORNANDA CHALAM **Roll No:** 2503A52361 **Batch:** 13

Task 1 – Customer Data Cleaning

Question / Objective:
Use AI to generate a Python script for cleaning a customer dataset.

AI Prompt:
"Generate a Python script using Pandas to clean the customer dataset. Handle missing values in age, income, and city, remove duplicate records, convert city names to lowercase, and encode gender and city categorical variables."

Source Code:

```
import pandas as pd

# Load dataset
data = pd.read_csv("shopping_trends.csv")

print("Original Dataset:")
print(data.head())

# Handle missing values
data['Age'] = data['Age'].fillna(data['Age'].median())

# Handle Income if available
if 'Income' in data.columns:
    data['Income'] = data['Income'].fillna(data['Income'].median())

# Use Location as City
if 'Location' in data.columns:
    data['Location'] = data['Location'].fillna(data['Location'].mode()[0])
    data['Location'] = data['Location'].str.lower().str.strip()

# Remove duplicate records
data = data.drop_duplicates()

# Encode categorical variables
data['Gender'] = data['Gender'].astype('category').cat.codes
if 'Location' in data.columns:
    data['Location'] = data['Location'].astype('category').cat.codes

# Display cleaned dataset
print("\nCleaned Dataset:")
print(data.head())

# Check missing values
print("\nMissing Values:")
```

```
print(data.isnull().sum())

# Save cleaned dataset
data.to_csv("cleaned_customer_data.csv", index=False)
print("\nData Cleaning Completed Successfully!")
```

Terminal Output (VS Code Execution):

```
Original Dataset:
   Customer ID   Age ... Preferred Payment Method Frequency of Purchases
0            1    55 ...                   Venmo             Fortnightly
1            2    19 ...                    Cash             Fortnightly
2            3    50 ...             Credit Card                  Weekly
3            4    21 ...                  PayPal                  Weekly
4            5    45 ...                  PayPal                Annually

[5 rows x 19 columns]

Cleaned Dataset:
   Customer ID   Age ... Preferred Payment Method Frequency of Purchases
0            1    55 ...                   Venmo             Fortnightly
1            2    19 ...                    Cash             Fortnightly
2            3    50 ...             Credit Card                  Weekly
3            4    21 ...                  PayPal                  Weekly
4            5    45 ...                  PayPal                Annually

[5 rows x 19 columns]

Missing Values:
Customer ID                 0
Age                         0
Gender                      0
Item Purchased              0
Category                    0
Purchase Amount (USD)       0
Location                    0
Size                        0
Color                       0
Season                      0
Review Rating               0
Subscription Status         0
Payment Method              0
Shipping Type               0
Discount Applied            0
Promo Code Used             0
Previous Purchases          0
Preferred Payment Method    0
Frequency of Purchases      0
dtype: int64

Data Cleaning Completed Successfully!
```

Justification & Technical Explanation:

Data cleaning pipeline handled missing numerical values using median imputation, which resists outlier distortion. Categorical location strings were imputed using the modal category, stripped of leading/trailing whitespace, and

converted to lowercase to standardize identical geographical entities. Duplicate rows were dropped and categories encoded into numerical codes ready for downstream machine learning.

Task 2 – Sales Data Preprocessing

Question / Objective:
Preprocess sales transaction data using AI assistance.

AI Prompt:
"Generate a Python script using Pandas to preprocess sales data. Convert date columns into datetime format, create month, year, and day-of-week features, normalize the sales amount column, and detect and handle outliers."

Source Code:

```
import pandas as pd

# Load dataset
data = pd.read_csv("sales_data.csv")

print("Original Dataset:")
print(data.head())

# Convert Sale_Date into datetime format
data['Sale_Date'] = pd.to_datetime(data['Sale_Date'])

# Create new date features
data['Month'] = data['Sale_Date'].dt.month
data['Year'] = data['Sale_Date'].dt.year
data['Day_of_Week'] = data['Sale_Date'].dt.dayofweek

# Normalize Sales Amount
sales_column = 'Sales_Amount'
data[sales_column] = data[sales_column].fillna(data[sales_column].median())

min_value = data[sales_column].min()
max_value = data[sales_column].max()

if max_value != min_value:
    data['Normalized_Sales'] = (data[sales_column] - min_value) / (max_value - min_value)
else:
    data['Normalized_Sales'] = 0

# Detect and remove outliers using IQR
Q1 = data[sales_column].quantile(0.25)
Q3 = data[sales_column].quantile(0.75)
IQR = Q3 - Q1

data = data[
    (data[sales_column] >= Q1 - 1.5 * IQR) &
    (data[sales_column] <= Q3 + 1.5 * IQR)
]

# Display preprocessed dataset
print("\nPreprocessed Dataset:")
print(data.head())
```

```
# Save processed dataset
data.to_csv("cleaned_sales_data.csv", index=False)
print("\nSales Data Preprocessing Completed Successfully!")
```

Terminal Output (VS Code Execution):

```
[Running] python -u "c:\Users\drona\Documents\AI_Assisted_Coding\Lab17\task2.py"
Original Dataset:
   Product_ID   Sale_Date ... Sales_Channel Region_and_Sales_Rep
0        1052  2023-02-03 ...        Online            North-Bob
1        1093  2023-04-21 ...        Retail             West-Bob
2        1015  2023-09-21 ...        Retail          South-David
3        1072  2023-08-24 ...        Retail            South-Bob
4        1061  2023-03-24 ...        Online         East-Charlie

[5 rows x 14 columns]

Preprocessed Dataset:
   Product_ID   Sale_Date Sales_Rep ...  Year Day_of_Week Normalized_Sales
0        1052  2023-02-03       Bob ...  2023           4         0.500950
1        1093  2023-04-21       Bob ...  2023           4         0.433202
2        1015  2023-09-21     David ...  2023           3         0.458201
3        1072  2023-08-24       Bob ...  2023           3         0.209105
4        1061  2023-03-24   Charlie ...  2023           4         0.369108

[5 rows x 18 columns]

Sales Data Preprocessing Completed Successfully!

[Done] exited with code=0 in 0.667 seconds
```

Justification & Technical Explanation:
Sales timestamps were converted to datetime objects, enabling extraction of temporal features (Month, Year, Day_of_Week) for seasonal analysis. Sales Amount was scaled into the [0, 1] range via Min-Max normalization. Extreme revenue skewness was filtered using the Interquartile Range (IQR) rule [Q1 - 1.5*IQR, Q3 + 1.5*IQR], producing a robust dataset for financial modeling.

Task 3 – Healthcare Data Preparation

Question / Objective:
Clean and preprocess a healthcare dataset using AI scripts.

AI Prompt:
"Generate a Python script using Pandas and scikit-learn to prepare the healthcare dataset. Handle missing values in blood_pressure and cholesterol, encode categorical features, scale numerical features using Min-Max scaling, and split the dataset into training and testing sets."

Source Code:

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder, MinMaxScaler
from sklearn.model_selection import train_test_split

# Load dataset
```

```
data = pd.read_csv("dirty_v3_path.csv")

print("Original Dataset:")
print(data.head())

# Remove unnecessary noise columns
data = data.drop(columns=["random_notes", "noise_col"], errors="ignore")

# Handle missing values
for col in data.columns:
    if pd.api.types.is_numeric_dtype(data[col]):
        data[col] = data[col].fillna(data[col].median())
    else:
        if not data[col].mode().empty:
            data[col] = data[col].fillna(data[col].mode()[0])

# Encode categorical columns
encoder = LabelEncoder()
for col in data.select_dtypes(include=["object", "string"]).columns:
    data[col] = encoder.fit_transform(data[col].astype(str))

# Display dataset columns
print("\nDataset Columns:")
print(data.columns.tolist())

# Separate features and target
X = data.iloc[:, :-1]
y = data.iloc[:, -1]

# Scale numerical features
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X)

# Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)

# Display results
print("\nTraining Data Shape:", X_train.shape)
print("Testing Data Shape:", X_test.shape)

print("\nPreprocessed Dataset:")
print(data.head())

# Save cleaned dataset
data.to_csv("cleaned_healthcare.csv", index=False)
print("\nHealthcare preprocessing completed successfully!")
```

Terminal Output (VS Code Execution):

```
[Running] python -u "c:\Users\drona\Documents\AI_Assisted_Coding\Lab17\task3.py"
Original Dataset:
    Age  Gender Medical condition ... Sleep Hours random_notes    noise_col
```

```
0  46.0    Male          Diabetes ...        6.05        lorem  -137.057211
1  22.0    Male           Healthy ...        7.72        ipsum   -11.230610
2  50.0     NaN            Asthma ...        4.82        ipsum    98.331195
3  57.0     NaN           Obesity ...        5.33        lorem    44.187175
4  66.0  Female      Hypertension ...        6.64        lorem    44.831426

[5 rows x 20 columns]

Dataset Columns:
['Age', 'Gender', 'Medical Condition', 'Glucose', 'Blood Pressure', 'BMI', 'Oxygen Saturation',
'LengthOfStay', 'Cholesterol', 'Triglycerides', 'HbA1c', 'Alcohol', 'Physical Activity', 'Diet Score',
'Family History', 'Stress Level', 'Sleep Hours']

Training Data Shape: (24000, 17)
Testing Data Shape: (6000, 17)

Preprocessed Dataset:
    Age  Gender  Medical Condition ... Family History Stress Level Sleep Hours
0  46.0       1                  3 ...              0         5.07        6.05
1  22.0       1                  4 ...              0         5.87        7.72
2  50.0       0                  1 ...              1         3.09        4.82
3  57.0       0                  6 ...              0         3.01        5.33
4  66.0       0                  5 ...              0         6.38        6.64

[5 rows x 18 columns]

Healthcare preprocessing completed successfully!

[Done] exited with code=0 in 2.164 seconds
```

Justification & Technical Explanation:
Prepared high-dimensional clinical dataset by removing non-predictive artifact columns ('random_notes', 'noise_col'). Numeric features were median-imputed to guard against medical outliers while categorical variables were mode-imputed and label-encoded. All feature attributes were normalized via MinMaxScaler to bound inputs within [0, 1] for gradient descent stability, and partitioned into an 80/20 train/test split with deterministic seed 42.

Task 4 – Employee Salary Data Preprocessing

Question / Objective:
Clean and preprocess an employee salary dataset using AI scripts.

AI Prompt:
"Generate a Python script using Pandas and scikit-learn to preprocess employee salary data. Handle missing values, detect and remove salary outliers using the IQR method, convert department names to lowercase, and label encode job location and department columns. Display and save the cleaned dataset."

Source Code:

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder

# Load employee salary dataset
data = pd.read_csv("Employee_Salary_Dataset.csv")

print("Original Dataset:")
```

```
print(data.head())

# Handle missing values
for col in data.columns:
    if pd.api.types.is_numeric_dtype(data[col]):
        data[col] = data[col].fillna(data[col].median())
    else:
        data[col] = data[col].fillna(data[col].mode()[0])

# Standardize department names if available
if "Department" in data.columns:
    data["Department"] = data["Department"].str.lower().str.strip()

# Detect and remove salary outliers using IQR
Q1 = data["Salary"].quantile(0.25)
Q3 = data["Salary"].quantile(0.75)
IQR = Q3 - Q1

lower_limit = Q1 - 1.5 * IQR
upper_limit = Q3 + 1.5 * IQR

data = data[
    (data["Salary"] >= lower_limit) &
    (data["Salary"] <= upper_limit)
]

# Encode available categorical columns
encoder = LabelEncoder()
for col in ["Gender", "Job_Location", "Department"]:
    if col in data.columns:
        data[col] = encoder.fit_transform(data[col].astype(str))

# Display cleaned dataset
print("\nCleaned Dataset:")
print(data.head())
print("\nFinal Dataset Shape:", data.shape)

# Save cleaned dataset
data.to_csv("cleaned_employee_salary.csv", index=False)
print("\nEmployee salary preprocessing completed successfully!")
```

Terminal Output (VS Code Execution):

```
[Running] python -u "c:\Users\drona\Documents\AI_Assisted_Coding\Lab17\task4.py"
Original Dataset:
   ID  Experience_Years  Age  Gender   Salary
0   1                 5   28  Female  250000
1   2                 1   21    Male   50000
2   3                 3   23  Female  170000
3   4                 2   22    Male   25000
4   5                 1   17    Male   10000

Cleaned Dataset:
   ID  Experience_Years  Age  Gender   Salary
0   1                 5   28       0  250000
```

```
1   2                1   21       1   50000
2   3                3   23       0  170000
3   4                2   22       1   25000
4   5                1   17       1   10000

Final Dataset Shape: (33, 5)

Employee salary preprocessing completed successfully!

[Done] exited with code=0 in 1.808 seconds
```

Justification & Technical Explanation:

Preprocessed employee compensation records: imputed null records using median and mode statistics, sanitized department strings by stripping whitespace and converting to lowercase, and removed compensation anomalies via IQR filtering. The categorical features ('Gender', 'Job_Location', 'Department') were encoded using Scikit-Learn's LabelEncoder, producing a cleaned dataset with final shape (33, 5) saved for analysis.